🧭 VIT Campus Navigation App Documentation

This document describes the structure and core functionalities of the VitNavigationApp, a Flutter application designed for indoor/campus navigation, integrating mapping, route finding, voice guidance, and real-time weather data.

1. Core Architecture

The application follows a **Modular State Management** pattern within a single StatefulWidget (_HomeScreenState), where global application state is managed. The use of DraggableScrollableSheet and custom AnimatedBuilder widgets optimizes UI performance by isolating map controls and the navigation bar from sheet scrolling logic.

Component	Responsibility	Key Libraries
HomeScreen (_HomeScreenState)	Manages all state: current location, destination, routing data, selected profile, and UI state (sheet position, selected tab). Contains the main FlutterMap widget.	Geolocator, flutter_map, flutter_tts, http
Location	Data model for campus points of interest (POIs).	latlong2
RouteStep	Data model for turn-by-turn navigation instructions.	latlong2
DirectoryScreen	Optimized screen for displaying the list of all campus locations using pre-processed data and a ListView.builder.	ListView.builder
CustomNavigationBar	The custom floating bottom bar for main screen navigation (Explore, Directory, Events, Profile).	StatelessWidget

2. Key Features

🗺️ Map & Location Services

Feature	Implementation Details
Mapping	Uses **FlutterMap** with **CachedTileProvider** for smooth, cached rendering of OpenStreetMap tiles.
Live Location	**Geolocator.getPositionStream** provides continuous location updates.
User Tracking	**CurrentLocationLayer** follows the user (FollowOnLocationUpdate.always) when _isFollowingUser is true. Rotates the map based on compass heading for a true navigation feel.
Destination Setting	A long press (onLongPress) on the map or selecting a location from the **Directory** sets the destination (_finalDestinationCoordinates).
Map Controls	The GPS button toggles **_isFollowingUser**. UI is optimized to hide map controls when the bottom sheet is up.

🚀 Routing & Navigation

Routing is handled by making HTTP requests to a local Open Source Routing Machine (OSRM) instance running on a private IP (100.81.0.57).

Feature	Function/Constant	Description
Routing Endpoints	_osrmWalkBaseUrl, _osrmBikeBaseUrl	Separate endpoints for the walking (/route/v1/walk) and biking (/route/v1/bike) profiles.
Route Calculation	_fetchRouteDetails(String profile)	Fetches both the polyline coordinates (_fullRouteCoordinates) and the turn-by-turn instructions (_activeSteps) from OSRM.
Turn-by-Turn	**_getManeuverIcon** and the **Navigation Card**	The bottom-positioned Card displays the current instruction (_activeSteps[_currentStepIndex].instruction) and the next upcoming

Feature	Function/Constant	Description
		step.
Voice Guidance	_speak(String text)	Uses **flutter_tts** to read out navigation instructions when a new step is reached (triggered by Geolocator updates near a maneuver point).
Off-Route Rerouting	Logic within the Geolocator.getPositionStream listener.	If the user drifts more than **_rerouteThreshold (30.0m)** from the expected polyline, a new route calculation is triggered, and a voice prompt is given.
Speedometer	_buildSpeedometerWidget	Displays real-time speed (_currentSpeed) derived from Position.speed during active navigation.

☀️ Weather Integration

The app fetches real-time weather and air quality data for the campus center from **Open-Meteo**.

Data Point	Function	Details
Weather/ Forecast	_fetchOpenMeteoWeather	Fetches temperature, weather code, hourly forecast, and min/max temperatures using the Open-Meteo **Forecast API**.
Air Quality (AQI)	_fetchOpenMeteoWeather	Fetches the US AQI index using the Open-Meteo **Air Quality API**.
Display	_buildWeatherWidget, _showWeatherDetailsSheet	A small widget in the header shows current temperature and AQI. Tapping it opens a detailed sheet showing full current conditions and a 12-hour hourly forecast.

3. UI and Performance Optimizations

📊 Performance Fixes

1. **Search & UI Decoupling (_buildAnimatedUI)**: A key performance optimization. The animated search bar and GPS button are now placed in a separate **AnimatedBuilder** that listens to the _sheetController. This prevents the entire HomeScreen widget tree from rebuilding on every minor drag gesture, resulting in a significantly smoother bottom sheet interaction.
2. **Pre-processing Directory Data (_preProcessDirectory)**: The complex task of grouping and sorting the _allLocations list by category is performed once in initState and stored in _directoryItems. This prevents heavy sorting logic from running on every rebuild of the Directory screen.
3. **Keyboard Lag Fix (Search Results)**: The search results list view now uses padding based on MediaQuery.of(context).viewInsets.bottom to adjust for the keyboard size, rather than relying on resizeToAvoidBottomInset: true on the Scaffold, which is set to false for the DraggableScrollableSheet's complex layout.

🌐 Destination Selector

The function **_setDestinationFromDirectory** includes advanced logic for multi-entry buildings:

- If a selected location has a buildingGroup (e.g., 'TT', 'MH-B'), the function checks for all listed entries belonging to that group (e.g., 'TT - Main Gate', 'TT - West Gate').
- It then calculates the **closest entry point** to the user's current position and uses that specific entry's coordinates as the final routing destination. This ensures the route directs the user to the most convenient entrance.